

✓ Step 7 — Apply Log Transformation

Why?

The raw `Price` is heavily right-skewed (skewness = 4.45).

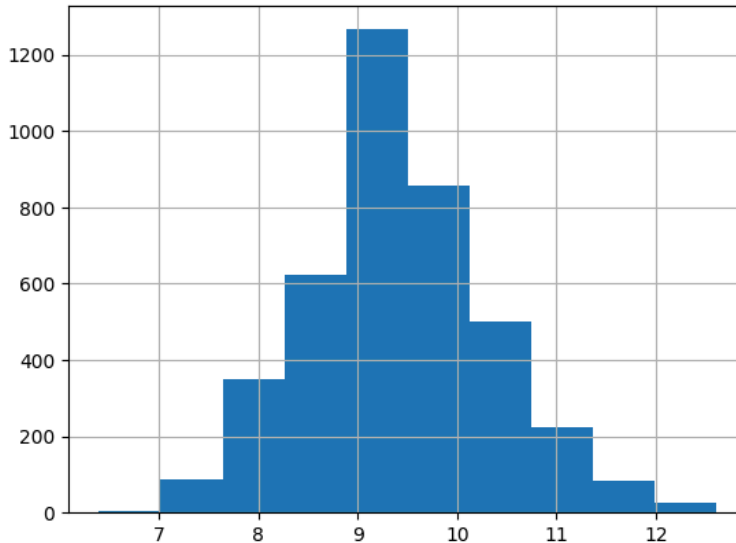
After `log(Price)`, skewness drops to ~0.24 — almost normal!

This helps our model make much better predictions.

⚠ We also drop the `Model` column — it has 312 unique values (too many for simple encoding).

```
df['Log_Price'] = np.log(df['Price'])
df = df.drop(columns=['Model'])
df['Log_Price'].hist()
```

<Axes: >



✓ Step 8 — Encode Categorical Columns

Why?

Machine learning models only understand **numbers**, not text like 'BMW' or 'Diesel'.

We use **Label Encoding** to convert text categories into numbers.

Text	Encoded
Audi	0
BMW	1
Toyota	2

💡 `LabelEncoder` assigns a unique number to each category automatically.

```
from sklearn.preprocessing import LabelEncoder

encoder = LabelEncoder()
cat_col = df.select_dtypes(include=['object']).columns
df[cat_col] = df[cat_col].apply(encoder.fit_transform)
df.head()
```

	Brand	Price	Body	Mileage	EngineV	Engine Type	Registration	Year	Log_Price	
0	1	4200.0	3	277	2.0	3	1	1991	8.342840	
1	2	7900.0	5	427	2.9	0	1	1999	8.974618	
2	2	13300.0	3	358	5.0	1	1	2003	9.495519	
3	0	23000.0	0	240	4.2	3	1	2007	10.043249	
4	5	18300.0	0	120	2.0	3	1	2011	9.814656	

Next steps: [Generate code with df](#) [New interactive sheet](#)

Step 9 — Correlation Heatmap

Why?

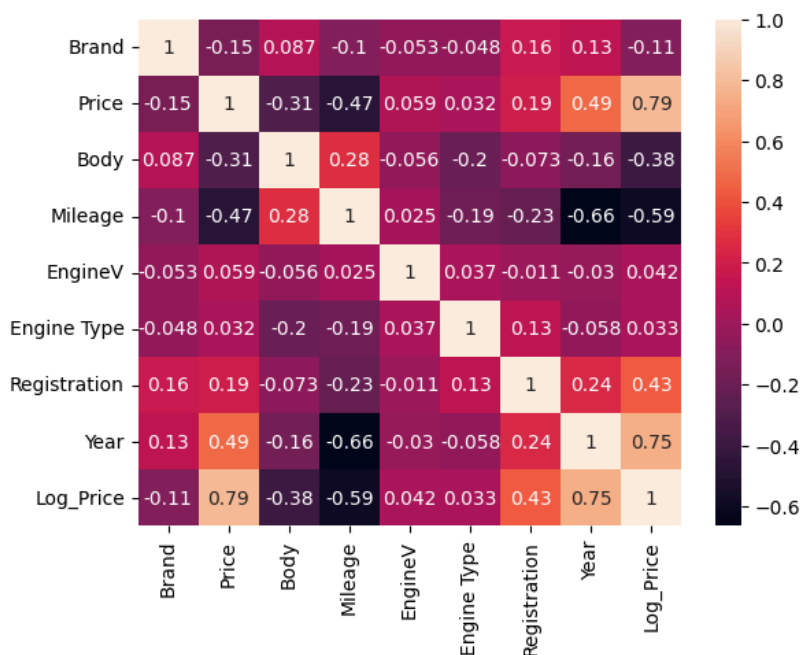
A heatmap shows how strongly each feature is related to `log_price`.

- Value close to **+1** → strong positive relationship (feature goes up, price goes up)
- Value close to **-1** → strong negative relationship (feature goes up, price goes down)
- Value close to **0** → weak/no relationship

💡 This helps us understand which features are most important for prediction.

```
sns.heatmap(df.corr(), annot=True)
```

<Axes: >



Step 10 — Split Data into Train and Test Sets

Why?

We train the model on one part of the data, and test it on a *different* part it has never seen.

This tells us how well the model performs on **new, unseen data** — which is the real goal!

Split	Purpose
80% Train	Model learns from this
20% Test	We evaluate model here



`random_state=42` ensures same split every time you run the code.

```
from sklearn.model_selection import train_test_split

X = df.drop(columns=['Log_Price'])
y = df['Log_Price']
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```